

Search Algorithms and Computational Complexity

From Mazes to Minimax and Beyond

Martin Benning

University College London

ENGF0034 – Design and Professional Skills

Lecture Overview

Objectives

Explore fundamental concepts of search algorithms, starting with single-agent environments (mazes) and progressing to adversarial environments (games). We will analyse the limitations of simple heuristics (Right-Hand Rule), the power of recursion and Depth-First Search (DFS), the optimality of the Minimax algorithm, and the challenges of computational complexity in complex domains like Chess.

Lecture Overview

- 1 Motivation: Understanding Fundamentals
- 2 Single-Agent Search: Mazes and Heuristics
- 3 Recursion and Depth-First Search (DFS)
- 4 Adversarial Search and Game Trees
- 5 The Minimax Algorithm
- 6 Implementation Analysis: The Board (oxo.py)
- 7 Implementation Analysis: The Minimax AI (oxo.py)
- 8 Testing and Analysis
- 9 Limitations and Advanced Techniques
- 10 Summary

Motivation: Understanding Fundamentals

Motivation: Understanding Fundamentals

The Philosophy of "The Elevator" (Alan Kay)

Alan Kay, Turing Award winner and pioneer of OOP and GUIs, often uses the metaphor of an elevator to distinguish between users and engineers.

The User Perspective

- Uses the elevator functionally: presses a button, arrives at a destination.
- Relies on abstraction. Does not need to understand the underlying mechanics.

The Engineer Perspective

- Understands the underlying principles (algorithms, data structures, physics).
- Capable of design, innovation, and solving fundamental problems.

The Philosophy of "The Elevator" (Alan Kay)

Alan Kay, Turing Award winner and pioneer of OOP and GUIs, often uses the metaphor of an elevator to distinguish between users and engineers.

The User Perspective

- Uses the elevator functionally: presses a button, arrives at a destination.
- Relies on abstraction. Does not need to understand the underlying mechanics.

The Engineer Perspective

- Understands the underlying principles (algorithms, data structures, physics).
- Capable of design, innovation, and solving fundamental problems.

The Philosophy of "The Elevator" (Alan Kay)

The User Perspective

- Uses the elevator functionally: presses a button, arrives at a destination.
- Relies on abstraction. Does not need to understand the underlying mechanics.

The Engineer Perspective

- Understands the underlying principles (algorithms, data structures, physics).
- Capable of design, innovation, and solving fundamental problems.

The Goal in Computer Science

We aim to be engineers. While abstraction is crucial, mastering the fundamental algorithms is essential for innovation. We must understand what happens inside the elevator shaft.

Single-Agent Search: Mazes and Heuristics

Single-Agent Search: Mazes and Heuristics

The Search Problem: Solving a Maze

We begin by examining search algorithms in a single-agent environment. The goal is to find a path from a Start (S) to an End (E) location.

The Challenge

Develop an algorithm that guarantees finding a solution if one exists, and ideally does so efficiently.

Hampton Court Palace Maze

A famous historical maze, often cited in discussions of maze-solving strategies (e.g., Jerome K. Jerome's *Three Men in a Boat*).

Can we rely on simple rules (heuristics) to solve arbitrary mazes?

The Search Problem: Solving a Maze

We begin by examining search algorithms in a single-agent environment. The goal is to find a path from a Start (S) to an End (E) location.

The Challenge

Develop an algorithm that guarantees finding a solution if one exists, and ideally does so efficiently.

Hampton Court Palace Maze

A famous historical maze, often cited in discussions of maze-solving strategies (e.g., Jerome K. Jerome's *Three Men in a Boat*).

Can we rely on simple rules (heuristics) to solve arbitrary mazes?

A Common Heuristic: The Right-Hand Rule (Wall Follower)

The Algorithm

- 1 Place your right hand on a wall (hedge) at the entrance.
- 2 Continuously follow the wall, maintaining contact with your right hand.

Application to Hampton Court

If applied consistently, the Right-Hand Rule successfully navigates the Hampton Court maze to the center, although the path taken is often significantly longer than the shortest path.

The Critical Question

Is this algorithm guaranteed to work for *all* mazes?

A Common Heuristic: The Right-Hand Rule (Wall Follower)

The Algorithm

- 1 Place your right hand on a wall (hedge) at the entrance.
- 2 Continuously follow the wall, maintaining contact with your right hand.

Application to Hampton Court

If applied consistently, the Right-Hand Rule successfully navigates the Hampton Court maze to the center, although the path taken is often significantly longer than the shortest path.

The Critical Question

Is this algorithm guaranteed to work for *all* mazes?

A Common Heuristic: The Right-Hand Rule (Wall Follower)

The Algorithm

- 1 Place your right hand on a wall (hedge) at the entrance.
- 2 Continuously follow the wall, maintaining contact with your right hand.

Application to Hampton Court

If applied consistently, the Right-Hand Rule successfully navigates the Hampton Court maze to the center, although the path taken is often significantly longer than the shortest path.

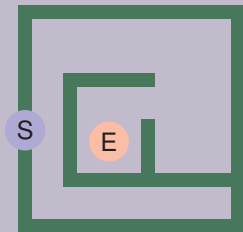
The Critical Question

Is this algorithm guaranteed to work for *all* mazes?

The Failure of the Right-Hand Rule: Cycles

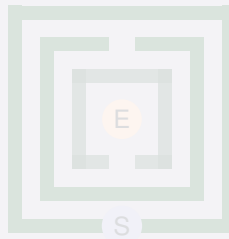
The Right-Hand Rule fails when maze topology contains cycles that isolate destination.

A Solvable Maze



Inner walls are connected to boundary. The RHR successfully navigates to E.

The Failure Case: An Island

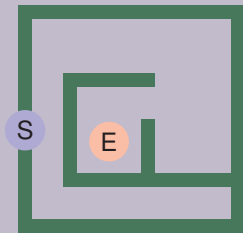


E is inside isolated inner rectangle. RHR follows connected walls, never reaching E.

The Failure of the Right-Hand Rule: Cycles

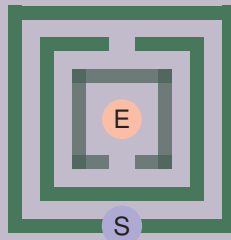
The Right-Hand Rule fails when maze topology contains cycles that isolate destination.

A Solvable Maze



Inner walls are connected to boundary. The RHR successfully navigates to E.

The Failure Case: An Island



E is inside isolated inner rectangle. RHR follows connected walls, never reaching E.

Topological Explanation: Connectivity

We can understand this limitation by analysing the maze's topology (the connectivity of its paths and walls).

The Condition for Wall Follower Success

The Right-Hand Rule (or any wall follower) is guaranteed to work **if and only if** the maze is **simply connected**.

Definition: Simply Connected Maze

A maze where all walls are connected to the outer boundary. There are no detached walls forming isolated islands or loops. The Hampton Court Maze is simply connected.

Topological Explanation: Connectivity

We can understand this limitation by analysing the maze's topology (the connectivity of its paths and walls).

The Condition for Wall Follower Success

The Right-Hand Rule (or any wall follower) is guaranteed to work **if and only if** the maze is **simply connected**.

Definition: Simply Connected Maze

A maze where all walls are connected to the outer boundary. There are no detached walls forming isolated islands or loops. The Hampton Court Maze is simply connected.

Topological Explanation: Connectivity

The Condition for Wall Follower Success

The Right-Hand Rule (or any wall follower) is guaranteed to work **if and only if** the maze is **simply connected**.

Definition: Simply Connected Maze

A maze where all walls are connected to the outer boundary. There are no detached walls forming isolated islands or loops. The Hampton Court Maze is simply connected.

The Implication

If the destination is surrounded by a detached cycle (making the maze multiply connected), a wall follower algorithm may become trapped in that cycle, unable to reach the destination. Heuristics are insufficient for general search problems.

The Need for a Guaranteed Algorithm

Since simple heuristics like the Right-Hand Rule can fail due to cycles, we require a more robust algorithm that can handle arbitrary maze topologies.

Requirements for a General Solver

- 1 **Termination:** The algorithm must not get stuck in infinite loops (must handle cycles).
- 2 **Completeness:** The algorithm must explore all reachable paths to guarantee finding the solution if it exists.

The Solution

This requires a systematic search strategy that maintains memory of the explored areas. The fundamental algorithm for this is **Depth-First Search (DFS)**, which relies on the principle of **Recursion**.

The Need for a Guaranteed Algorithm

Since simple heuristics like the Right-Hand Rule can fail due to cycles, we require a more robust algorithm that can handle arbitrary maze topologies.

Requirements for a General Solver

- 1 **Termination:** The algorithm must not get stuck in infinite loops (must handle cycles).
- 2 **Completeness:** The algorithm must explore all reachable paths to guarantee finding the solution if it exists.

The Solution

This requires a systematic search strategy that maintains memory of the explored areas. The fundamental algorithm for this is **Depth-First Search (DFS)**, which relies on the principle of **Recursion**.

Recursion and Depth-First Search (DFS)

Recursion and Depth-First Search (DFS)

What is Recursion?

Definition

Recursion is a method of solving a computational problem where the solution depends on solutions to simpler or previous instances of the same problem. In programming, this means a function that calls itself.

The Structure of a Recursive Function

Every correct recursive implementation requires two essential components:

1. Base Case: A termination scenario that does not use recursion to produce an answer.
2. Recursive Step: A set of rules that reduces all successive cases toward the base case.

What is Recursion?

Definition

Recursion is a method of solving a computational problem where the solution depends on solutions to simpler or previous instances of the same problem. In programming, this means a function that calls itself.

The Structure of a Recursive Function

Every correct recursive implementation requires two essential components:

1. **Base Case:** A termination scenario that does not use recursion to produce an answer.
2. **Recursive Step:** A set of rules that reduces all successive cases toward the base case.

The Mechanism: The Call Stack

Recursion relies heavily on program's call stack to manage execution flow and local state.

How it Works

- 1 When a function is called (including recursively), a new **Stack Frame** is pushed onto the call stack, storing local variables and the return address.
- 2 The recursion "winds" as frames are pushed until the base case is reached.
- 3 The recursion "unwinds" as the base case returns, and frames are popped off the stack sequentially.

Implications

The call stack acts as the memory for the search process. The space complexity is proportional to the maximum depth of the recursion ($\mathcal{O}(D)$).

The Mechanism: The Call Stack

Recursion relies heavily on program's call stack to manage execution flow and local state.

How it Works

- 1 When a function is called (including recursively), a new **Stack Frame** is pushed onto the call stack, storing local variables and the return address.
- 2 The recursion "winds" as frames are pushed until the base case is reached.
- 3 The recursion "unwinds" as the base case returns, and frames are popped off the stack sequentially.

Implications

The call stack acts as the memory for the search process. The space complexity is proportional to the maximum depth of the recursion ($\mathcal{O}(D)$).

Applying Recursion to Maze Solving (DFS)

We can design a guaranteed maze solver using recursion. This approach naturally implements Depth-First Search (DFS).

The Recursive Insight

A path exists from the current location L to the End E **if and only if** a path exists from one of L 's adjacent, reachable neighbors to E .

Addressing the Cycle Problem (Requirement 1: Termination)

To prevent infinite loops, the algorithm must track its history. We must augment the state space.

Strategy: Mark locations as "Visited" when entering them (dropping breadcrumbs).

Applying Recursion to Maze Solving (DFS)

We can design a guaranteed maze solver using recursion. This approach naturally implements Depth-First Search (DFS).

The Recursive Insight

A path exists from the current location L to the End E **if and only if** a path exists from one of L 's adjacent, reachable neighbors to E .

Addressing the Cycle Problem (Requirement 1: Termination)

To prevent infinite loops, the algorithm must track its history. We must augment the state space.

Strategy: Mark locations as "Visited" when entering them (dropping breadcrumbs).

The DFS Maze Algorithm

The algorithm systematically explores the maze while preventing cycles.

Algorithm: Solve(Location)

- 1 **Base Case 1 (Success):** If Location is End, return 'True'.
- 2 **Base Case 2 (Failure):** If Location is Wall or **Visited**, return 'False'. (This prevents cycles).
- 3 **State Update:** Mark Location as **Visited**.
- 4 **Recursive Step:** For each Neighbor (e.g., N, E, S, W):
 - If 'Solve(Neighbor)' returns 'True', return 'True'.
- 5 **Exhaustion (Dead End):** If all neighbors fail, return 'False'.

The Mechanism of Backtracking (Requirement 2: Completeness)

What happens when the algorithm reaches a dead end and returns 'False' (Step 5)?

Backtracking

When a recursive call returns 'False', the execution returns to the previous stack frame (the previous location). The algorithm then attempts the next unexplored direction from that location (the loop in Step 4 continues).

Completeness Guaranteed

This process of deep exploration and subsequent retraction, managed implicitly by the call stack, ensures that every reachable path is eventually explored. This guarantees finding a solution if one exists.

The Mechanism of Backtracking (Requirement 2: Completeness)

What happens when the algorithm reaches a dead end and returns 'False' (Step 5)?

Backtracking

When a recursive call returns 'False', the execution returns to the previous stack frame (the previous location). The algorithm then attempts the next unexplored direction from that location (the loop in Step 4 continues).

Completeness Guaranteed

This process of deep exploration and subsequent retraction, managed implicitly by the call stack, ensures that every reachable path is eventually explored. This guarantees finding a solution if one exists.

The Mechanism of Backtracking (Requirement 2: Completeness)

What happens when the algorithm reaches a dead end and returns 'False' (Step 5)?

Completeness Guaranteed

This process of deep exploration and subsequent retraction, managed implicitly by the call stack, ensures that every reachable path is eventually explored. This guarantees finding a solution if one exists.

Path Reconstruction (Refinement)

To reconstruct the actual path, or allow a square to be part of an alternative path later, we must "unmark" the cell (change "Visited" back to "Open") when backtracking from a dead end.

Adversarial Search and Game Trees

Adversarial Search and Game Trees

From Single-Agent to Adversarial Search

In maze solving (DFS), we seek an optimal path in a passive environment. We control all the decisions. In Adversarial Search (Games), the environment is hostile.

The New Challenge

We face rational opponents actively working against our goals. We need a **strategy** that accounts for the opponent's moves, assuming the opponent also plays optimally.

Focus: Two-Player Zero-Sum Games

We will study games with the following properties:

- **Two-Player** and **Zero-Sum** (One player's gain is the other's loss).
- **Deterministic** (No chance) and **Perfect Information** (Full state visible).

From Single-Agent to Adversarial Search

In maze solving (DFS), we seek an optimal path in a passive environment. We control all the decisions. In Adversarial Search (Games), the environment is hostile.

The New Challenge

We face rational opponents actively working against our goals. We need a **strategy** that accounts for the opponent's moves, assuming the opponent also plays optimally.

Focus: Two-Player Zero-Sum Games

We will study games with the following properties:

- **Two-Player** and **Zero-Sum** (One player's gain is the other's loss).
- **Deterministic** (No chance) and **Perfect Information** (Full state visible).

From Single-Agent to Adversarial Search

The New Challenge

We face rational opponents actively working against our goals. We need a **strategy** that accounts for the opponent's moves, assuming the opponent also plays optimally.

Focus: Two-Player Zero-Sum Games

We will study games with the following properties:

- **Two-Player** and **Zero-Sum** (One player's gain is the other's loss).
- **Deterministic** (No chance) and **Perfect Information** (Full state visible).

Examples

Tic-Tac-Toe (Noughts and Crosses), Chess, Checkers, Go.

Modeling Games: The Game Tree

The fundamental structure for analysing these games is the Game Tree, which is analogous to the search space of the maze.

Definition

A Game Tree represents all possible sequences of moves from the starting state.

- **Nodes:** Represent game states (board configurations).
- **Edges:** Represent moves (actions).
- **Root:** The initial state.
- **Leaves (Terminal Nodes):** States where the game ends (Win/Loss/Draw).

Modeling Games: The Game Tree

The fundamental structure for analysing these games is the Game Tree, which is analogous to the search space of the maze.

Definition

A Game Tree represents all possible sequences of moves from the starting state.

- **Nodes:** Represent game states (board configurations).
- **Edges:** Represent moves (actions).
- **Root:** The initial state.
- **Leaves (Terminal Nodes):** States where the game ends (Win/Loss/Draw).

Modeling Games: The Game Tree

Definition

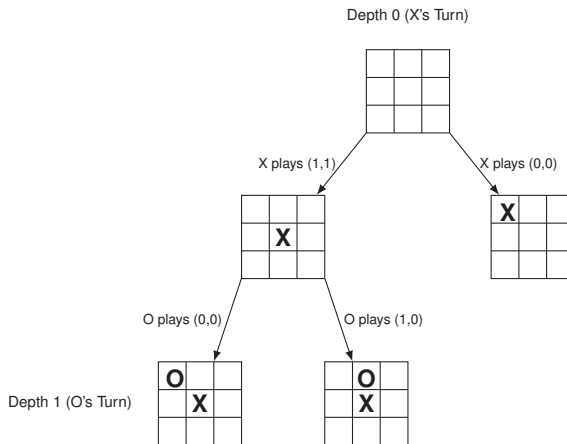
A Game Tree represents all possible sequences of moves from the starting state.

- **Nodes:** Represent game states (board configurations).
- **Edges:** Represent moves (actions).
- **Root:** The initial state.
- **Leaves (Terminal Nodes):** States where the game ends (Win/Loss/Draw).

The Connection to DFS

Just as DFS explores the maze, we use a variant of DFS to explore the Game Tree. However, the decisions at alternating levels are controlled by different players.

A Partial Tic-Tac-Toe Game Tree



Game Complexity and State Space

The feasibility of solving a game depends on the size of its game tree.

Metrics

- b : Branching factor (average number of legal moves per turn).
- d : Depth (maximum length of the game in turns/ply).
- Game Tree Size (Time Complexity of Search): $\mathcal{O}(b^d)$.

Tic-Tac-Toe Analysis

- $d = 9$. $b \leq 9$.
- Naive Upper Bound: $9! = 362,880$ possible move sequences.
- Optimized Count: Considering early terminations (wins) and symmetries, the number of reachable game states is much smaller ($\approx 52,592$ unique game paths).
- Conclusion: This is computationally trivial. We can exhaustively search the entire tree.

Game Complexity and State Space

The feasibility of solving a game depends on the size of its game tree.

Metrics

- b : Branching factor (average number of legal moves per turn).
- d : Depth (maximum length of the game in turns/ply).
- Game Tree Size (Time Complexity of Search): $\mathcal{O}(b^d)$.

Tic-Tac-Toe Analysis

- $d = 9$. $b \leq 9$.
- Naive Upper Bound: $9! = 362,880$ possible move sequences.
- Optimized Count: Considering early terminations (wins) and symmetries, the number of reachable game states is much smaller ($\approx 52,592$ unique game paths).
- **Conclusion:** This is computationally trivial. We can exhaustively search the entire tree

Game Complexity and State Space

Metrics

- b : Branching factor (average number of legal moves per turn).
- d : Depth (maximum length of the game in turns/ply).
- Game Tree Size (Time Complexity of Search): $\mathcal{O}(b^d)$.

Tic-Tac-Toe Analysis

- $d = 9$. $b \leq 9$.
- Naive Upper Bound: $9! = 362,880$ possible move sequences.
- Optimized Count: Considering early terminations (wins) and symmetries, the number of reachable game states is much smaller ($\approx 52,592$ unique game paths).
- **Conclusion:** This is computationally trivial. We can exhaustively search the entire tree and "solve" the game.

The Challenge of Exponential Complexity

The $\mathcal{O}(b^d)$ complexity leads to a combinatorial explosion for more complex games.

The Case of Chess

- Branching Factor $b \approx 35$ (often approximated as 30).
- Typical Depth $d \approx 100$ (50 moves per side).
- Estimated Game Tree Size: $\approx 35^{100}$ (or 30^{50} approximated).

The Scale of Intractability

$30^{50} \approx 7 \times 10^{73}$. Calculating tree would take $\approx 4.33 \times 10^{57}$ seconds with current computing power (age of universe $\approx 4.35 \times 10^{17}$ seconds).

The Challenge of Exponential Complexity

The $\mathcal{O}(b^d)$ complexity leads to a combinatorial explosion for more complex games.

The Case of Chess

- Branching Factor $b \approx 35$ (often approximated as 30).
- Typical Depth $d \approx 100$ (50 moves per side).
- Estimated Game Tree Size: $\approx 35^{100}$ (or 30^{50} approximated).

The Scale of Intractability

$30^{50} \approx 7 \times 10^{73}$. Calculating tree would take $\approx 4.33 \times 10^{57}$ seconds with current computing power (age of universe $\approx 4.35 \times 10^{17}$ seconds).

The Challenge of Exponential Complexity

The Case of Chess

- Branching Factor $b \approx 35$ (often approximated as 30).
- Typical Depth $d \approx 100$ (50 moves per side).
- Estimated Game Tree Size: $\approx 35^{100}$ (or 30^{50} approximated).

The Scale of Intractability

$30^{50} \approx 7 \times 10^{73}$. Calculating tree would take $\approx 4.33 \times 10^{57}$ seconds with current computing power (age of universe $\approx 4.35 \times 10^{17}$ seconds).

Conclusion

Exhaustive search is impossible for Chess. We need a different approach for complex games. For Tic-Tac-Toe, however, it is the perfect tool.

The Minimax Algorithm

The Minimax Algorithm

The Goal: Optimal Play Against a Rational Opponent

We want an algorithm that chooses the best possible move, assuming the opponent also plays optimally to defeat us.

Defining Utility and Players

We assign numerical utilities to terminal states from perspective of primary player (MAX).

MAX: Tries to maximise the final utility score (e.g., +1 for Win).

MIN: The opponent tries to minimise the final utility score (e.g., -1 for Loss).

Draw: Utility of 0.

The Minimax Principle

"Maximise your minimum possible gain." MAX chooses the move that maximises the final score, assuming MIN will subsequently play to minimise that score.

The Goal: Optimal Play Against a Rational Opponent

We want an algorithm that chooses the best possible move, assuming the opponent also plays optimally to defeat us.

Defining Utility and Players

We assign numerical utilities to terminal states from perspective of primary player (MAX).

MAX: Tries to maximise the final utility score (e.g., +1 for Win).

MIN: The opponent tries to minimise the final utility score (e.g., -1 for Loss).

Draw: Utility of 0.

The Minimax Principle

"Maximise your minimum possible gain." MAX chooses the move that maximises the final score, assuming MIN will subsequently play to minimise that score.

The Goal: Optimal Play Against a Rational Opponent

We want an algorithm that chooses the best possible move, assuming the opponent also plays optimally to defeat us.

Notation

$V(s)$ Minimax value of state s from MAX's perspective.

$Utility(s)$ Terminal utility for MAX (e.g., +1 win, 0 draw, -1 loss).

$Player(s)$ Player to move in s , either MAX or MIN.

$Successors(s)$ Set of legal next states reachable from s by one legal move.

The Minimax Recurrence Relation

The Minimax value of a state s , $V(s)$, is defined recursively, utilising DFS to explore the game tree. The tree is often built dynamically during the search.

Recursive Definition

$$V(s) = \begin{cases} \text{Utility}(s) & \text{if } s \text{ is terminal (Base Case)} \\ \max_{s' \in \text{Successors}(s)} V(s') & \text{if Player}(s) = \text{MAX (Recursive Step)} \\ \min_{s' \in \text{Successors}(s)} V(s') & \text{if Player}(s) = \text{MIN (Recursive Step)} \end{cases}$$

The Algorithm in Action

Utilities are calculated at the leaves (the end of the game) and propagated upwards through the tree. This is analogous to the 'True'/'False' values propagated back in the DFS maze

The Minimax Recurrence Relation

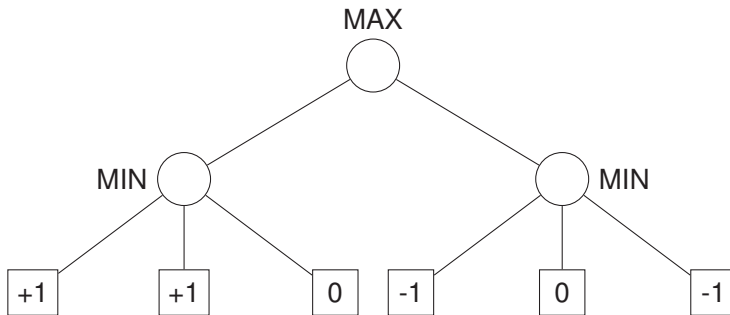
Recursive Definition

$$V(s) = \begin{cases} \text{Utility}(s) & \text{if } s \text{ is terminal (Base Case)} \\ \max_{s' \in \text{Successors}(s)} V(s') & \text{if Player}(s) = \text{MAX (Recursive Step)} \\ \min_{s' \in \text{Successors}(s)} V(s') & \text{if Player}(s) = \text{MIN (Recursive Step)} \end{cases}$$

The Algorithm in Action

Utilities are calculated at the leaves (the end of the game) and propagated upwards through the tree. This is analogous to the 'True'/'False' values propagated back in the DFS maze solver, but adapted for the adversarial environment.

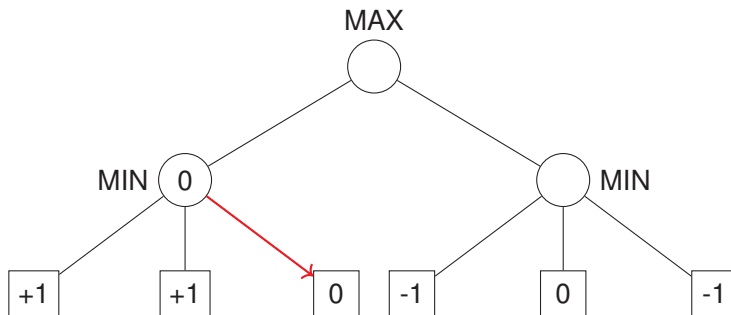
Visualising Minimax Propagation



The Result of Optimal Play

By applying Minimax to entire Tic-Tac-Toe game tree, we determine that value of initial state is 0. This proves that with optimal play from both sides, the game always ends in a draw.

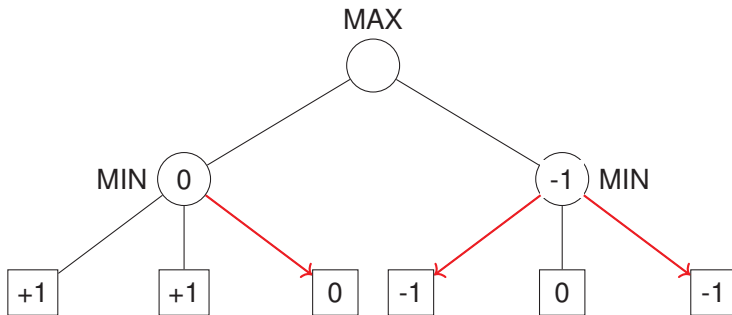
Visualising Minimax Propagation



The Result of Optimal Play

By applying Minimax to entire Tic-Tac-Toe game tree, we determine that value of initial state is 0. This proves that with optimal play from both sides, the game always ends in a draw.

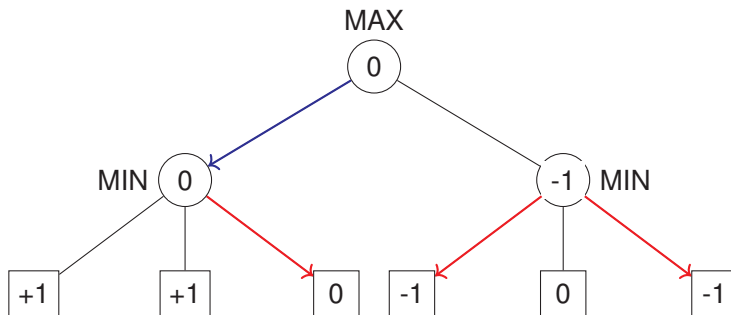
Visualising Minimax Propagation



The Result of Optimal Play

By applying Minimax to entire Tic-Tac-Toe game tree, we determine that value of initial state is 0. This proves that with optimal play from both sides, the game always ends in a draw.

Visualising Minimax Propagation



The Result of Optimal Play

By applying Minimax to entire Tic-Tac-Toe game tree, we determine that value of initial state is 0. This proves that with optimal play from both sides, the game always ends in a draw.

Implementation Analysis: The Board (oxo.py)

Implementation Analysis: The Board (oxo.py)

The Structure of the Implementation

The provided `oxo.py` separates the game mechanics (the rules) from the strategy (the AI). This is good design (Separation of Concerns).

The Board Class

Responsibilities: Maintain the state of the 3x3 grid and implement the rules (moves, win/draw checks).

The `oxo` Class (The AI Engine)

Responsibilities: Implement the Minimax search algorithm.

The Importance of Efficiency

The 'Board' operations must be highly efficient as they are called repeatedly during the Minimax search (potentially hundreds of thousands of times).

The Structure of the Implementation

The provided `oxo.py` separates the game mechanics (the rules) from the strategy (the AI). This is good design (Separation of Concerns).

The Board Class

Responsibilities: Maintain the state of the 3x3 grid and implement the rules (moves, win/draw checks).

The `oxo` Class (The AI Engine)

Responsibilities: Implement the Minimax search algorithm.

The Importance of Efficiency

The 'Board' operations must be highly efficient as they are called repeatedly during the Minimax search (potentially hundreds of thousands of times).

The Structure of the Implementation

The provided `oxo.py` separates the game mechanics (the rules) from the strategy (the AI). This is good design (Separation of Concerns).

The Board Class

Responsibilities: Maintain the state of the 3x3 grid and implement the rules (moves, win/draw checks).

The `oxo` Class (The AI Engine)

Responsibilities: Implement the Minimax search algorithm.

The Importance of Efficiency

The 'Board' operations must be highly efficient as they are called repeatedly during the Minimax search (potentially hundreds of thousands of times).

Board Representation and Core Operations I

Listing 1: Board Initialisation and Operations (oxo.py)

```
1 class Board:
2     def reset_board(self):
3         # A list of lists (2D array)
4         self.pieces = [['.', '.', '.'],
5                         ['.', '.', '.'],
6                         ['.', '.', '.']]
7         self.count = 0 # number of pieces already played
8
9         # is there still somewhere to play (O(1))
10    def unfinished(self):
11        return self.count < 9
```

Board Representation and Core Operations II

Design Choice

‘self.count’ is maintained to efficiently detect draws (when count reaches 9) in $\mathcal{O}(1)$ time, avoiding a full board scan ($\mathcal{O}(N^2)$ for an $N \times N$ board).

The Critical Operation: Win Detection I

The 'wins(player)' method checks the 8 possible winning lines.

Listing 2: Board.wins (oxo.py)

```
1  def wins(self, player):
2      colcounts = [0,0,0]
3      d1count, d2count = 0, 0
4      for y in range(0,3):
5          rowcount = 0
6          for x in range(0,3):
7              if self.pieces[y][x] == player:
8                  rowcount+=1
9                  colcounts[x] += 1
10                 if colcounts[x] == 3: return True
```

The Critical Operation: Win Detection II

```
11         if x == y: # Main diagonal
12             d1count += 1
13             if d1count == 3: return True
14         if x == 2 - y: # Anti-diagonal
15             d2count += 1
16             if d2count == 3: return True
17         if rowcount == 3: return True
18     return False
```

The Critical Operation: Win Detection III

Algorithm Analysis

This implementation is optimised. It iterates through the 9 cells exactly once, simultaneously accumulating counts for all potential winning lines. Complexity is $\mathcal{O}(1)$ (constant time, as the board size is fixed).

Test-Driven Development (TDD) and `test_board.py` I

Before implementing complex recursive AI, we must ensure fundamental rules are correct.

The Role of `test_board.py`

Provides unit tests to verify the functional correctness of the `Board` class in isolation, particularly the crucial `wins()` method.

Listing 3: Excerpt from `test_board.py`

```
1 def test_board():  
2     board = Board()  
3     # Test Case: Main Diagonal Win  
4     board.set("X..",  
5               ".X.",
```

Test-Driven Development (TDD) and test_board.py II

```
6         "..X")
7     assert(board.wins("X"))
8
9     # Test Case: Vertical Win
10    board.set("0..",
11              "0..",
12              "0..")
13    assert(board.wins("0"))
14    assert(not board.wins("X"))
```

Implementation Analysis: The Minimax AI (oxo.py)

Implementation Analysis: The Minimax AI (oxo.py)

Implementation Strategy: Symbolic Outcomes

The 'oxo.py' implementation uses symbolic constants instead of numerical utilities (+1, -1, 0).

Listing 4: Outcome Constants (oxo.py)

```
1 WIN_IMMEDIATE = 0 # Win on this turn
2 WIN = 1          # Win eventually (forced win)
3 LOSE = 2         # Lose eventually (assuming optimal opponent)
4 DRAW = 3
```

Equivalence to Minimax

This is functionally equivalent. The algorithm prioritises outcomes according to the following preference relation, which replaces the max/min operators:

$$\text{WIN_IMMEDIATE} > \text{WIN} > \text{DRAW} > \text{LOSE}$$

Handling State During Search: The Need for Cloning

When exploring hypothetical moves during the recursive search (building the dynamic game tree), we must not modify the current game state.

The Problem: Backtracking Complexity

If we modify the board in-place, we must "undo" the move before backtracking to try the next alternative. This is complex and error-prone.

The Solution: Cloning (Deep Copy)

Create an independent copy of the game state before simulating a move.

Listing 5: Oxo.clone (oxo.py)

```
1 from copy import deepcopy
2 # ...
3 class Oxo:
4     # ...
```

Handling State During Search: The Need for Cloning

The Problem: Backtracking Complexity

If we modify the board in-place, we must "undo" the move before backtracking to try the next alternative. This is complex and error-prone.

The Solution: Cloning (Deep Copy)

Create an independent copy of the game state before simulating a move.

Listing 6: Oxo.clone (oxo.py)

```
1 from copy import deepcopy
2 # ...
3 class Oxo:
4     # ...
5     def clone(self):
6         return deepcopy(self)
```

Handling State During Search: The Need for Cloning

Why deepcopy is Essential

The Board object contains mutable structures (lists of lists). A shallow copy would only copy references. `deepcopy` ensures the cloned board's pieces are fully independent, preventing side effects during the search.

The Recursive Structure: Mutual Recursion

The Minimax algorithm is implemented using **mutual recursion** between two methods in the `Oxo` class. This models the alternating turns of the MAX and MIN players.

`choose_best_move(player)`: Iterates over possible moves and selects the best one (The MAX role).

`test_move(pos, player)`: Evaluates a specific move by recursively calling `choose_best_move` for the opponent (Simulating the MIN role).



The Recursive Structure: Mutual Recursion

The Minimax algorithm is implemented using **mutual recursion** between two methods in the `Oxo` class. This models the alternating turns of the MAX and MIN players.

`choose_best_move(player)`: Iterates over possible moves and selects the best one (The MAX role).

`test_move(pos, player)`: Evaluates a specific move by recursively calling `choose_best_move` for the opponent (Simulating the MIN role).

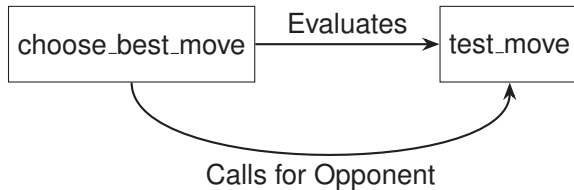


The Recursive Structure: Mutual Recursion

The Minimax algorithm is implemented using **mutual recursion** between two methods in the `Oxo` class. This models the alternating turns of the MAX and MIN players.

`choose_best_move(player)`: Iterates over possible moves and selects the best one (The MAX role).

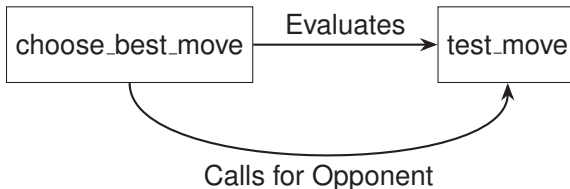
`test_move(pos, player)`: Evaluates a specific move by recursively calling `choose_best_move` for the opponent (Simulating the MIN role).



The Recursive Structure: Mutual Recursion

`choose_best_move(player)`: Iterates over possible moves and selects the best one (The MAX role).

`test_move(pos, player)`: Evaluates a specific move by recursively calling `choose_best_move` for the opponent (Simulating the MIN role).



Dynamic Tree Generation

The game tree is generated dynamically as these functions call each other recursively down to the terminal nodes.

The Recursive Core: test_move (I)

This function evaluates the consequence of a specific move.

Listing 7: Oxo.test_move (oxo.py)

```
1  def test_move(self, pos, player):
2      # 1. Clone the state and simulate the move
3      game = self.clone()
4      game.board.play(pos, player)
5      # 2. Base Case 1: Check for immediate win
6      if game.board.wins(player):
7          return WIN_IMMEDIATE
8      # 3. Recursive Step: Evaluate the opponent's best response
9      # This assumes the opponent will play optimally (MIN logic)
10     result = game.choose_best_move(other_player(player))
11     # 4. Interpret the result (The Minimax inversion)
12     # ... (continued) ...
```

The Recursive Core: test_move (II) - The Adversarial Flip I

Listing 8: Oxo.test_move - Interpretation (oxo.py)

```
79 # 4. Interpret the result (The Minimax inversion)
80 if result == WIN:
81     # If the opponent's best outcome is a WIN, this move is
      a LOSS for us
82     return LOSE
83 elif result == LOSE:
84     # If the opponent's best outcome is a LOSS, this move
      is a WIN for us
85     return WIN
86 return DRAW
```

The Recursive Core: `test_move` (II) - The Adversarial Flip II

The Zero-Sum Logic

This inversion (WIN becomes LOSE, LOSE becomes WIN) perfectly captures the adversarial, zero-sum nature of the game. The value of a move is the inverse of the best outcome the opponent can achieve from the resulting state. This implements the MIN step of the recurrence.

The Driver: choose_best_move I

This function iterates over all possible moves and selects the one that yields the best Minimax outcome (The MAX role).

Listing 9: Oxo.choose_best_move (oxo.py)

```
1 def choose_best_move(self, player):
2     # Base Case 2: Board is full (Draw)
3     if self.board.unfinished() == False:
4         return DRAW
5     # Initialize variables to track the best move found so far
6     winning_move = None; drawing_move = None; losing_move =
       None
7
8     # Iterate over all empty squares
```

The Driver: choose_best_move II

```
9     for y in range(0,3):
10         for x in range(0,3):
11             # ... (Check if empty) ...
12             # Evaluate the move using the recursive helper
13             result = self.test_move((x,y), player)
14             # Optimization: Short-circuit on immediate win
15             if result == WIN_IMMEDIATE:
16                 self.board.play((x,y), player)
17                 return WIN
18             # Record the move based on its outcome
19             if result == WIN: winning_move = (x,y)
20             # ... (Record DRAW/LOSE) ...
```


The Driver: choose_best_move (II) - Selection I

Listing 10: Oxo.choose_best_move - Selection (oxo.py)

```
66     # Select the best outcome found (Prioritize Win > Draw >
        Lose)
67     if winning_move:
68         self.board.play(winning_move, player)
69         return WIN
70     if drawing_move:
71         self.board.play(drawing_move, player)
72         return DRAW
73     if losing_move:
74         self.board.play(losing_move, player)
75         return LOSE
```

The Driver: `choose_best_move` (II) - Selection II

The MAX Logic

The final selection implements the MAX preference (the maximisation step of the recurrence): prioritise a guaranteed Win, settle for a Draw if necessary, and accept a Loss only if unavoidable.

Testing and Analysis

Testing and Analysis

Testing the Strategy (test_oxo.py) I

We use unit tests to verify the AI makes optimal decisions in specific scenarios. This is crucial for validating complex recursive algorithms.

Listing 11: Excerpt from test_oxo.py

```
1 def test_gameplay():
2     game = Oxo()
3     # Scenario 1: Immediate Win Available
4     game.board.set("X0.",
5                   "X0.",
6                   "...")
7     # X should play at (0,2) to win immediately
8     assert(game.choose_best_move("X") == WIN)
9
```

Testing the Strategy (test_oxo.py) II

```
10 # Scenario 2: Proving the Optimal Outcome
11 game.board.set("...", "...", "...")
12 # The optimal result from the start is a draw
13 assert(game.choose_best_move("X") == DRAW)
```

Minimax Complexity Analysis

Properties

Completeness: Yes, if the game tree is finite (like Tic-Tac-Toe).

Optimality: Yes, it guarantees the optimal strategy against a rational opponent.

Complexity

Time Complexity: $\mathcal{O}(b^d)$. It performs a complete DFS of the game tree.

Space Complexity: $\mathcal{O}(d)$. Proportional to the depth of the tree (for the recursion stack).

The Fundamental Limitation

As established earlier, the exponential time complexity $\mathcal{O}(b^d)$ makes pure Minimax intractable for complex games like Chess.

Minimax Complexity Analysis

Properties

Completeness: Yes, if the game tree is finite (like Tic-Tac-Toe).

Optimality: Yes, it guarantees the optimal strategy against a rational opponent.

Complexity

Time Complexity: $\mathcal{O}(b^d)$. It performs a complete DFS of the game tree.

Space Complexity: $\mathcal{O}(d)$. Proportional to the depth of the tree (for the recursion stack).

The Fundamental Limitation

As established earlier, the exponential time complexity $\mathcal{O}(b^d)$ makes pure Minimax intractable for complex games like Chess.

Minimax Complexity Analysis

Properties

Completeness: Yes, if the game tree is finite (like Tic-Tac-Toe).

Optimality: Yes, it guarantees the optimal strategy against a rational opponent.

Complexity

Time Complexity: $\mathcal{O}(b^d)$. It performs a complete DFS of the game tree.

Space Complexity: $\mathcal{O}(d)$. Proportional to the depth of the tree (for the recursion stack).

The Fundamental Limitation

As established earlier, the exponential time complexity $\mathcal{O}(b^d)$ makes pure Minimax intractable for complex games like Chess.

Limitations and Advanced Techniques

Limitations and Advanced Techniques

Dealing with Intractability

Since we cannot search the entire game tree for complex games (e.g., Chess, 35^{100}), we must modify the strategy.

The Core Problem

Pure Minimax requires searching all the way to the terminal nodes to calculate utilities.

The Solution: Approximate Optimality

We must accept that we cannot find the truly optimal move. Instead, we aim for the best move possible within computational constraints. This involves two key concepts: Bounded Search Horizons and Heuristics.

Dealing with Intractability

Since we cannot search the entire game tree for complex games (e.g., Chess, 35^{100}), we must modify the strategy.

The Core Problem

Pure Minimax requires searching all the way to the terminal nodes to calculate utilities.

The Solution: Approximate Optimality

We must accept that we cannot find the truly optimal move. Instead, we aim for the best move possible within computational constraints. This involves two key concepts: Bounded Search Horizons and Heuristics.

Strategy 1: Bounded Search Horizon (Depth Limit)

The Approach

Limit the depth of the Minimax search to a fixed horizon h .

Chess Example

If we limit the search to $h = 6$ plays (3 moves for each side).

- Complexity reduces from $\mathcal{O}(35^{100})$ to $\mathcal{O}(35^6)$.
- 35^6 is computationally feasible.

The New Challenge

If we stop the search at depth h , the nodes at the horizon are typically not terminal states. We do not know their true utility (Win/Loss/Draw).

Strategy 1: Bounded Search Horizon (Depth Limit)

The Approach

Limit the depth of the Minimax search to a fixed horizon h .

Chess Example

If we limit the search to $h = 6$ plays (3 moves for each side).

- Complexity reduces from $\mathcal{O}(35^{100})$ to $\mathcal{O}(35^6)$.
- 35^6 is computationally feasible.

The New Challenge

If we stop the search at depth h , the nodes at the horizon are typically not terminal states. We do not know their true utility (Win/Loss/Draw).

Strategy 1: Bounded Search Horizon (Depth Limit)

The Approach

Limit the depth of the Minimax search to a fixed horizon h .

Chess Example

If we limit the search to $h = 6$ plays (3 moves for each side).

- Complexity reduces from $\mathcal{O}(35^{100})$ to $\mathcal{O}(35^6)$.
- 35^6 is computationally feasible.

The New Challenge

If we stop the search at depth h , the nodes at the horizon are typically not terminal states. We do not know their true utility (Win/Loss/Draw).

Strategy 2: Heuristic Evaluation Functions

To apply Minimax with a bounded horizon, we need a way to estimate the value of non-terminal states.

Heuristic Evaluation Function $E(s)$

A function that takes a game state s and returns a numerical estimate of its strategic value (how "good" the position is for MAX). This replaces the $Utility(s)$ function in the Minimax recurrence for nodes at the horizon.

Designing Heuristics

Heuristics incorporate domain knowledge. They must be fast to calculate ($\mathcal{O}(1)$ relative to the search).

Strategy 2: Heuristic Evaluation Functions

To apply Minimax with a bounded horizon, we need a way to estimate the value of non-terminal states.

Heuristic Evaluation Function $E(s)$

A function that takes a game state s and returns a numerical estimate of its strategic value (how "good" the position is for MAX). This replaces the $Utility(s)$ function in the Minimax recurrence for nodes at the horizon.

Designing Heuristics

Heuristics incorporate domain knowledge. They must be fast to calculate ($\mathcal{O}(1)$ relative to the search).

Strategy 2: Heuristic Evaluation Functions

Designing Heuristics

Heuristics incorporate domain knowledge. They must be fast to calculate ($\mathcal{O}(1)$ relative to the search).

Chess Material Heuristic

A simple heuristic is based on material advantage.

- Pawn=1, Knight/Bishop=3, Rook=5, Queen=9, King=1,000,000 (representing checkmate).
- $E(s) = (\text{Sum of MAX's pieces}) - (\text{Sum of MIN's pieces})$.

Modern engines use highly complex heuristics involving positional factors, king safety, etc.

Advanced Technique: Quiescence Search

Bounded search introduces the **Horizon Effect**: the algorithm might miss a significant event just beyond the search depth h .

The Horizon Problem

Engine searches to depth h and evaluates the position as good. However, at depth $h + 1$, a catastrophic move (e.g., losing the Queen) occurs. The bounded search is blind to this.

Quiescence Search

A technique to mitigate the horizon effect by extending the search selectively.

- If a position at the horizon is "volatile" or "active" (e.g., immediate captures are possible, checks), the search continues until the position becomes "quiet" (quiescent).
- This ensures that the evaluation function is applied to stable positions, improving accuracy without the cost of increasing the depth uniformly.

Optimization: Alpha-Beta Pruning

Even with bounded horizons, the search space is large. Can we avoid exploring branches that will not influence the final decision? Yes.

The Principle of Pruning

If we have already found a move that guarantees a certain outcome, we do not need to explore other moves that we can already determine will lead to a worse outcome.

Intuitive Example

MAX has found a move A guaranteeing a score of 5. MAX is now evaluating move B. If MAX realises that MIN has a response to B that leads to a score of 3, MAX can immediately stop evaluating B (prune). Since $3 < 5$, move B will not be chosen.

Impact

Optimization: Alpha-Beta Pruning

Even with bounded horizons, the search space is large. Can we avoid exploring branches that will not influence the final decision? Yes.

The Principle of Pruning

If we have already found a move that guarantees a certain outcome, we do not need to explore other moves that we can already determine will lead to a worse outcome.

Intuitive Example

MAX has found a move A guaranteeing a score of 5. MAX is now evaluating move B. If MAX realises that MIN has a response to B that leads to a score of 3, MAX can immediately stop evaluating B (prune). Since $3 < 5$, move B will not be chosen.

Impact

Optimization: Alpha-Beta Pruning

The Principle of Pruning

If we have already found a move that guarantees a certain outcome, we do not need to explore other moves that we can already determine will lead to a worse outcome.

Intuitive Example

MAX has found a move A guaranteeing a score of 5. MAX is now evaluating move B. If MAX realises that MIN has a response to B that leads to a score of 3, MAX can immediately stop evaluating B (prune). Since $3 < 5$, move B will not be chosen.

Impact

Alpha-Beta Pruning is essential. In best case (optimal move ordering), it reduces effective branching factor from b to $\sqrt{b}$, allowing engine to search twice as deep within same time.

Summary

Summary

Lecture Summary I

Single-Agent Search (Mazes)

- Simple heuristics (Right-Hand Rule) fail if the topology includes cycles (not simply connected).
- Guaranteed solutions require systematic search: Depth-First Search (DFS).
- DFS uses memory (visited sets) to prevent cycles and backtracking to ensure completeness.
- DFS is naturally implemented via recursion.

Adversarial Search (Games)

Lecture Summary II

- Modeled using Game Trees. Complexity is exponential $\mathcal{O}(b^d)$.
- **Minimax Algorithm:** The foundation for optimal play in zero-sum games (MAX maximizes, MIN minimizes). Implemented via recursive DFS.
- Exhaustive Minimax solves simple games (e.g., Tic-Tac-Toe results in a Draw).

Implementation and Complexity Management

Lecture Summary III

- Implementation (oxo.py) uses mutual recursion to model alternating turns. Correct state management ('deepcopy') is crucial.
- For complex games (Chess), exhaustive search is intractable.
- We manage complexity using Bounded Depth Search, Heuristic Evaluation Functions, Quiescence Search, and Alpha-Beta Pruning.